An Automatic HCPN Modeling Method for Microservice-Orchestrated Network Operation Workflows

Guoshuai Li
Inner Mongolia University
Hohhot, Inner Mongolia, China
32309213@mail.imu.edu.cn

Tao Sun
Inner Mongolia University
Hohhot, Inner Mongolia, China
cssunt@imu.edu.cn

Wenjie Zhong
Inner Mongolia University
Hohhot, Inner Mongolia, China
zhongwenjie@mail.imu.edu.cn

Abstract

Microservice architecture, with distributed deployment and loose coupling, is widely used in networked service systems. As the number of service modules grows and cross-service interactions become complex, formal modeling and verification face challenges. Hierarchical Colored Petri Net (HCPN)–based model checking can verify full execution paths, but manual modeling is costly, error-prone, and prone to state-space explosion. This paper proposes an automated HCPN modeling method that transforms Netflix Conductor workflow specifications into hierarchical HCPN models using predefined rules. The method preserves workflow structure, control flow, and data dependencies, and generates models suitable for formal verification. Experimental results show that the generated HCPN models are consistent with the original workflows and can automatically detect errors such as data inconsistency and business–data state inconsistency. Compared with existing methods, this approach reduces manual effort, lowers modeling cost, and provides more comprehensive support for data representation and error detection.

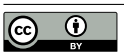
Keywords

Microservice, Network Modeling, Hierarchical Colored Petri Net (HCPN), Automatic Model Construction, Model Checking

1 Introduction

In recent years, microservice architecture has become an important paradigm for networked service systems. By decomposing complex functions into lightweight, loosely coupled services, microservices support flexible deployment, independent evolution, and elastic scaling[4]. System functions are often organized as workflows composed of multiple interacting services rather than monolithic programs.

However, microservice workflows involve service invocation, asynchronous communication, data transmission, conditional branching, concurrent execution, and sub-workflow calls, which make their behavior complex and uncertain[10]. Traditional testing and runtime verification methods (e.g., unit testing, integration testing, runtime monitoring) cannot cover all possible execution paths, leaving defects like data inconsistency, missing data, and unexpected state coupling undetected before deployment[3, 7, 9].



This work is licensed under a Creative Commons Attribution 4.0 International License.
SIGCOMM '26, Denver, CO, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2467-1/26/08
https://doi.org/10.1145/3789240.3822357

Formal model checking can explore the system state space and verify desired properties. Petri nets describe concurrency, synchronization, and causal relations, while Colored Petri Nets (CPNs)[6] introduce color sets, variables, and data expressions to model control and data flows. Hierarchical Colored Petri Nets (HCPNs)[5] extend CPNs with substitution transitions and subpages, enabling modular representation of complex workflows[11].

Constructing formal models directly from source code is difficult due to heterogeneous languages, frameworks, and low-level details irrelevant to workflow logic, which inflates model size and state space. In contrast, the TBPN method[2] based on microservice orchestration specifications automatically generates Petri Net from Netflix Conductor. Although these specifications provide a structured, high-level description of tasks, input-output mappings, decision branches, parallel tasks, and sub-workflows, and preserve key control- and data-flow relations, the generated PN models lack data semantics.

To address this, this paper proposes an automated HCPN modeling method for microservice workflows. The method transforms Conductor specifications into HCPN models by extracting task parameters, expressions, and execution logic, mapping tasks to HCPN fragments, and assembling them into hierarchical structures via substitution transitions and subpage matching. The resulting models can be directly opened and verified in CPN Tools.

The main contributions of this work are as follows:

1. We propose an automated HCPN modeling method for microservice orchestration workflows in networked service systems. By transforming Netflix Conductor workflow specifications into formal HCPN models, the proposed method reduces manual modeling efforts and improves modeling efficiency.

2. The generated HCPN models can be directly verified in CPN Tools using ASK-CTL formulas, enabling effective detection of workflow-level defects such as data inconsistency, missing data, and unexpected state coupling.

3. Compared with source-code-based coarse-fine-grained modeling methods, the proposed automated modeling method operates at a higher level of abstraction. While ensuring correct core business data propagation, it omits low-level implementation details not directly related to the workflow logic. Consequently, it significantly reduces the number of model elements and mitigates the state-space explosion problem, thereby substantially improving the feasibility of model checking for large-scale workflows.

The remainder of this paper is organized as follows. Section 2 introduces preliminary concepts. Section 3 presents the proposed automated HCPN modeling method. Section 4 reports the experimental results and analysis. Section 5 concludes the paper.

2 Preliminaries

This section introduces the background knowledge underlying the proposed approach. It first outlines the fundamentals of Hierarchical Colored Petri Nets (HCPNs), followed by an overview of Netflix Conductor's orchestration specification, and finally defines the categories of data transmission errors analyzed in this study.

2.1 HCPN Preliminaries

Definition 1. Colored Petri Net (CPN) [6]. A Colored Petri Net (CPN) is a mathematical modeling formalism used to represent both the control flow and the data flow of concurrent systems. Formally, a CPN is defined as a nine-tuple $CPN = (P, T, A, \Sigma, V, C, G, E, I)$, where:

P is the set of *places*, representing system states or data storage locations; T is the set of *transitions*, representing actions or events that change states; A is the set of *arcs*, connecting places and transitions; Σ is the set of *color sets*, defining the data types of tokens; V is the set of *variables* used in arc and guard expressions; $C : P \to \Sigma$ assigns a color set to each place; $G : T \to \mathrm{Expr}(V)$ assigns guard conditions to transitions; $E : A \to \mathrm{Expr}(V)$ assigns expressions to arcs, describing token transformations; $I : P \to \mathrm{Expr}(V)$ defines the initial marking of each place.

A Hierarchical Colored Petri Net (HCPN) [5] thus extends the flat CPN by introducing substitution transitions, port places, and socket places, enabling modular and scalable model construction: a substitution transition corresponds to a subprocess (a subpage), while port and socket places provide the linkage between hierarchical pages. An HCPN can therefore represent nested workflow structures while preserving data-flow and control-flow semantics across different pages.

2.2 Netflix Conductor Microservice Specification

Netflix Conductor is a workflow orchestration engine designed for distributed microservice coordination. It defines task execution sequences, conditional branches, and error-handling strategies through workflow templates written in JSON or DSL formats [1]. Each workflow template consists of multiple tasks, where each task is represented as a set of key–value pairs describing its attributes. Table 1 summarizes the key attributes relevant to this study.

Table 1: Key attributes of tasks in the workflow orchestration

Key	Description
name	Task name
taskReferenceName	Reference identifier
type	Task type definition
inputParameters	Input parameters for task execution
outputParameters	Output parameters produced by the task
taskDefinition	Task implementation expression
${task.xx}	Dynamic reference to data from other tasks

The mapping focuses on the fields listed in Table 1; other user-defined fields are treated as auxiliary metadata and excluded from transformation. The type field determines the transformation rule for each task: functional types (HTTP, SIMPLE, SCRIPT, SUB_WORK FLOW, EVENT) are mapped to transitions with input/output places, while control types (FORK, JOIN, DECISION, EXCLUSIVE_JOIN, WAIT) are represented as specialized transitions with guards for branching and synchronization. This mapping defines the expressiveness boundary of the proposed Conductor-to-HCPN transformation.

2.3 Data Error Categories

Referring to the typical abnormal patterns used in existing error-detection studies on microservice systems [8, 12], this paper selects categories of errors as detection targets: data inconsistency and business–data state inconsistency. These errors reflect typical error scenarios in microservice orchestration workflows.

- **Data Inconsistency.** Data inconsistency occurs when the data passed between preceding and subsequent tasks are inconsistent in semantics or values.
- **Business–Data State Inconsistency.** Business–data state inconsistency occurs when the business execution state changes, but the related data state is not updated or restored accordingly.

3 Automatic modeling methods

3.1 Overview of the Method

The proposed method automatically constructs HCPN models from Netflix Conductor workflow specifications. As shown in Figure 1, the method takes a JSON-based workflow specification as input and generates an HCPN model that can be directly used for model checking in CPN Tools.

The overall process consists of four main steps. First, the workflow specification is parsed to extract task information, input and output parameters, data references, and control-flow structures. Second, these extracted elements are organized into intermediate representations, including parameter lists, task expression lists, and task logic trees. Third, predefined transformation rules are applied to convert different types of Conductor tasks into corresponding HCPN model fragments. Finally, the generated fragments are assembled into a hierarchical HCPN model through substitution transitions and subpage matching.

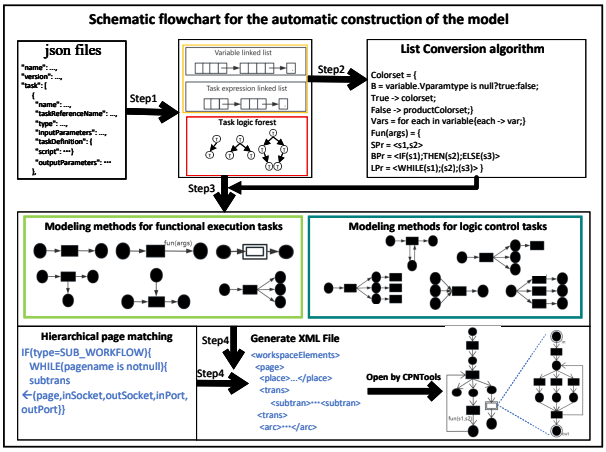


Figure 1: Overview of the proposed method.

3.2 Workflow Preprocessing

During the preprocessing stage, the Conductor workflow specification is analyzed to extract the key information required for model construction. Specifically, the workflow parser performs the following three main operations:

1. Parses the input and output parameters in the workflow and stores this parameter information as a parameter linked list.

2. Analyses tasks involving computation or data transformation to extract their expressions. These expressions are stored in a task expression list.

3. Analyses the execution logic structure of the workflow, constructing the task execution logic into a task logic tree, with relevant information about the tasks stored in the tree nodes. This representation preserves the hierarchical structure and dependencies between tasks.

This structured data stores the data and control flow information within the workflow. It serves as the foundation for generating HCPN fragments and assembling them into a complete hierarchical model, which is intended to reflect the execution logic and data dependencies of the original workflow.

3.3 Construction of Basic Model Elements

The parameter linked list is converted into CPN color sets and variable declarations. For each unique primary type, we create a base or product color set. Traversing the list identifies used color set names, generates definitions, and produces variable declarations for arc annotations and guards.

Color sets capture token data types. Derived from input/output parameters, tokens at specific positions explicitly represent service data, forming the basis for verifying data consistency and propagation during service call transformation.

Task expressions are parsed and converted into SML functions for arcs. Since they are stored as strings, parsing extracts key fields and generates complex arc expressions per statement type. For assignment statements, they are split and described using let-in-end. For if-else constructs, they are decomposed and described using if-then-else with function calls. For switch-case, a case-of structure with function calls is used. For loop statements (for, while, do-while), not directly supported, the conversion uses if-then-else with recursive calls to achieve equivalent transformation.

Arc expressions encode data transformations during task execution. Converting task scripts to SML precisely models output computation from inputs, key for checking data consistency and detecting erroneous data propagation.

3.4 Construction of Model Fragments

1. Functional-execution tasks model fragments. When functional execution tasks are converted into an HCPN model, they are typically mapped to a model fragment consisting of a single transition and multiple places. This mapping method is primarily used to describe changes in data state before and after task execution, as well as the input-output relationships during task processing. Figure 2 provides examples of HCPN model fragments corresponding to different functional execution tasks. Here, D denotes the color set bound to the place; data input is represented by directed arcs (PtoT) pointing from the input place to the transition, whilst data output is represented by directed arcs (TtoP) pointing from the transition to the output place.

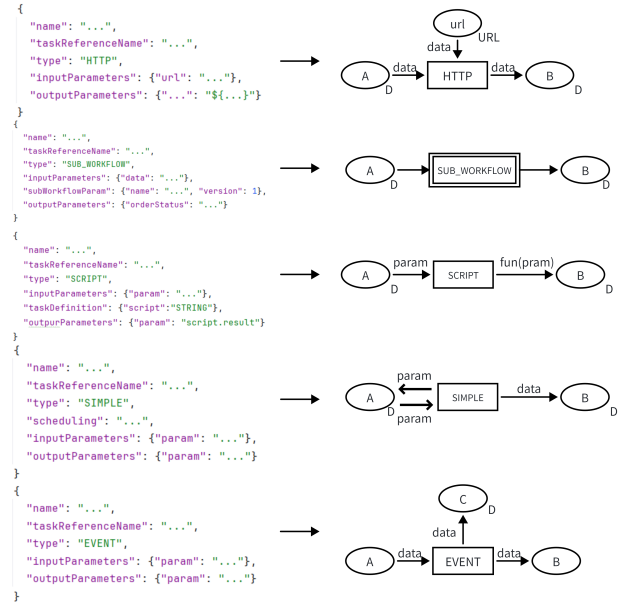


Figure 2: Functional Execution Class Task Conversion Rules.

2. Process control class tasks model fragments. Structural control tasks do not directly handle specific business logic calculations; rather, they are used to organise and schedule the execution relationships of functional execution tasks, thereby expressing process behaviours such as branch selection, concurrent distribution, synchronous convergence, and termination control. Their core function lies in constraining and organising the control flow structure between tasks, thereby ensuring that different business tasks are executed in an orderly manner according to the expected process logic. Figure 3 respectively illustrates the mapping rules for HCPN model fragments corresponding to different structural control tasks; tasks nested within these control structures may be either functional execution tasks or structural control tasks of a deeper level.

3.5 Model Fragment Assembly and Hierarchical Structure

Once the model fragments have been generated, the system proceeds to concatenate and assemble them into the corresponding model pages, based on the sequence, branches and parallel execution relationships defined in the task logic tree, thereby ensuring that the page-level model provides a comprehensive representation of the execution logic within the current module.

Each workflow tree is transformed into one or more pages: the main page represents the top-level flow, and SUB_WORKFLOW tasks are initially modeled as substitution transitions linking to subpages. The proposed method constructs an association table (mapping substitution transition IDs to subpage IDs and port/socket bindings) to implement hierarchization. First, the method attempts

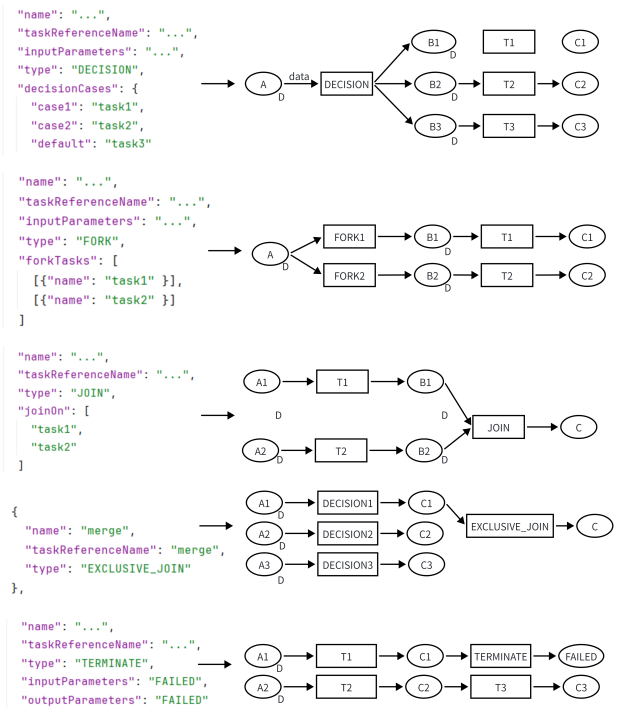


Figure 3: Process control class task conversion rules.

to match sub-workflow names with the generated subpages to establish port bindings, which represent explicit interface invocations within the system and route data across pages through the established socket-port mappings. If a direct matching subpage is not found, existing model pages sharing identical prefixes are merged to represent different interfaces integrated within the same service module. Subsequently, a secondary matching process is executed using the association table. Upon a successful match, this merged page becomes the valid subpage corresponding to the substitution transition on the main page. This procedure achieves a complete and consistent hierarchization of the model.

substitution transitions and port/socket bindings preserve hierarchical execution semantics: tokens entering a substitution transition are forwarded to the corresponding subpage ports, where internal transitions consume and produce tokens; upon subpage completion tokens return to the parent page. This mechanism precisely models nested workflow invocation and maintains correctness checks across page boundaries (type checking on ports and arc expressions).

4 Practical Application Examples

This section uses an online shopping mall system as a case study to validate the correctness and effectiveness of the method proposed in this paper, focusing in particular on the correctness of the automatically generated HCPN model, its anomaly detection capabilities, the effectiveness of model element and state space reduction, and its performance in comparison with related Petri net modeling methods.

4.1 Case Study

The overall structure of the constructed model is illustrated in Figure 4. This model is automatically generated using the automated modeling method proposed in this paper; however, it should be noted that the visual layout of the presented elements has been manually adjusted for better readability. At the center lies the main page, which depicts the interaction paths among all service modules. The upper-left subpage corresponds to the userservice module, linked to the substitution transition of the same name. The left subpage represents the itemservice module, while the lower-left subpage corresponds to the cartservice module. On the lower right is the tradeservice module page, and on the upper right lies the payservice module page. Each module page is a subpage corresponding to its substitution transition. The complete model can be opened in CPN Tools for analysis and verification using ASK-CTL formulas.

4.2 Model Correctness Verification

In this section, we analyse model correctness using an online shopping mall system. The generated HCPN models are validated against source-code-based coarse- and fine-grained models by comparing verification results. To avoid state-space explosion, the reference model only retains data relevant to the current validation.

Two representative business scenarios—user login and order creation—are selected. The login scenario verifies whether the model captures basic business execution logic, while the order creation scenario checks the accuracy of data transfer relationships in a more complex workflow.

For the user login function in the userservice module, model verification results (Figure 5) show that both models satisfy the ASK-CTL formula and return the variable binding for user 'jack'. This confirms that the proposed method correctly represents the execution logic of the user login scenario.

Next, the order creation function in the tradeservice module is verified. As shown in Figure 6, the ASK-CTL formula returns 'true' for both the proposed and reference models, and both yield the same variable bindings for the queried product, indicating successful order creation. Compared to the login scenario, this test further confirms that the generated model correctly represents data transfer relationships and bindings. Overall, the results demonstrate that the automatically generated model is consistent with the reference model in both business logic and data flow, validating its correctness.

4.3 Model Validity Verification

Error Detection Effectiveness. To validate the effectiveness of error detection, this paper refers to typical anomaly types utilized in existing microservice error-detection research, selecting two categories of errors as detection targets: data inconsistency and business-data state inconsistency. These anomalies respectively correspond to data value exceptions during cross-service data transmission and synchronization failures between business execution and data states, reflecting typical error scenarios in microservice orchestration workflows. To conduct the evaluation, these errors are manually introduced into the orchestration files, and the corresponding HCPN models are automatically generated to detect them.

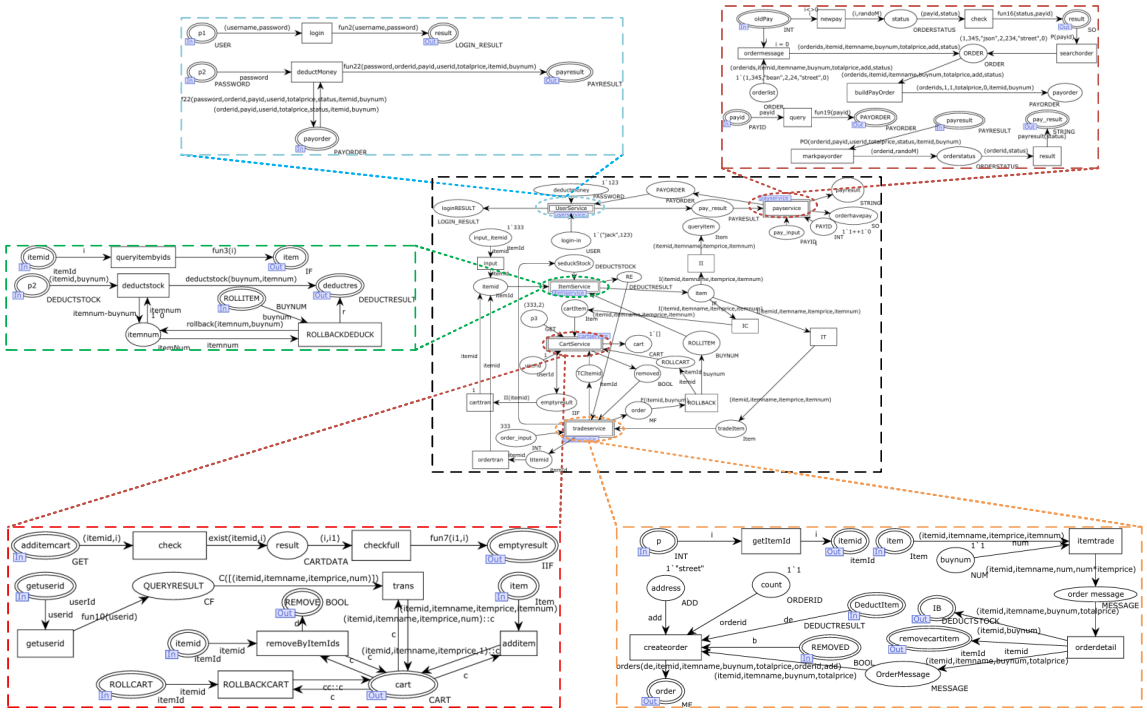


Figure 4: HCPN Model of Online Shopping Mall

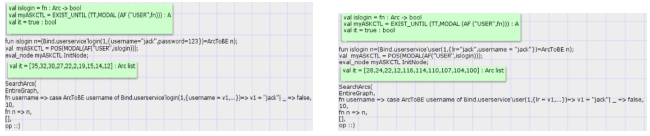


Figure 5: UserLogin ASK-CTL formula model checking.

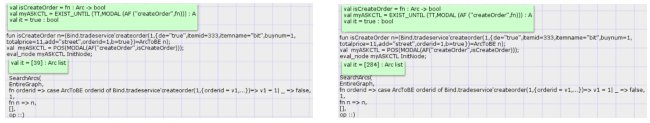


Figure 6: CreateOrder ASK-CTL formula model checking.



Figure 7: Data inconsistency model checking results.

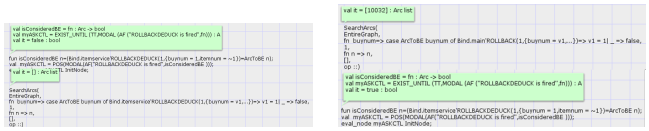


Figure 8: Business and data status consistency model detection results.

For data inconsistency, the product query in the itemservice workflow was modified so that itemid=333 returns the result for itemid=345. Under normal operation, the ASK-CTL formula returns true for itemid=333; after the modification, it returns false, showing that the model effectively identifies data inconsistency during cross-task data transfer (Figure 7).

To verify business–data state discrepancies, the rollback process in the order payment stage was removed. Without rollback, product stock and shopping basket data remain incorrect despite order cancellation. The model detects this discrepancy (Figure 8), while including rollback restores consistency. These results demonstrate that the generated models can effectively detect both data inconsistencies and business–data state discrepancies.

Model and State-Space Reduction. This section evaluates the effectiveness of the proposed method in reducing model size and mitigating state-space explosion. We compared models generated by our method with source-code-based coarse- and fine-grained models across microservice systems of 2, 4, 6, and 10 modules, adjusting token counts for each scale.

Tables 2–4 present the results. Model element reduction rates for nodes, transitions, and arcs range from 76% to 82%, 80% to 83%, and 74% to 78%, respectively, demonstrating substantial size reduction. For state-space metrics, direct comparisons were possible under small-scale configurations, showing reductions of approximately 76% for both nodes and arcs. Furthermore, as detailed in the preceding tables for other larger-scale configurations, the proposed method demonstrates a distinct advantage in effectively computing the state space under identical experimental conditions. These results indicate that the proposed method consistently reduces model complexity and alleviates the state-space explosion problem.

Table 2: Model and state-space scalability statistics.

Configuration	Model Elements		State Space	
Modules/Workflows	Tokens	Places/Trans./Arcs	SS Nodes/Arcs	Secs/Status
2/4	7	26 / 13 / 40	294 / 847	1 / Full
2/4	12	26 / 13 / 40	2240 / 8474	1 / Full
2/4	17	26 / 13 / 40	31307 / 220822	300 / Partial
4/10	8	42 / 24 / 80	852 / 3124	1 / Full
4/10	13	42 / 24 / 80	4128 / 16168	3 / Full
4/10	18	42 / 24 / 80	31917 / 214885	300 / Partial
6/12	8	79 / 39 / 124	6842 / 24650	5 / Full
6/12	16	79 / 39 / 124	21617 / 92295	8 / Full
6/12	24	79 / 39 / 124	30777 / 219053	300 / Partial
10/25	31	125 / 61 / 221	32214 / 235127	300 / Partial

Table 3: Coarse-fine grained model and state space scalability statistics

Configuration	Model Elements		State Space	
Modules/Workflows	Tokens	Places/Trans./Arcs	SS Nodes/Arcs	Secs/Status
2/4	7	109 / 64 / 163	1,245 / 3,542	1 / Full
2/4	12	109 / 64 / 163	21,241 / 205,108	300 / Partial
2/4	17	109 / 64 / 163	22,567 / 215,135	300 / Partial
4/10	8	235 / 132 / 357	20,210 / 188,450	300 / Partial
4/10	13	235 / 132 / 357	22,414 / 188,312	300 / Partial
4/10	18	235 / 132 / 357	31,123 / 236,805	300 / Partial
6/12	8	373 / 212 / 481	23,682 / 228,517	300 / Partial
6/12	16	373 / 212 / 481	21,206 / 215,614	300 / Partial
6/12	24	373 / 212 / 481	23,509 / 231,223	300 / Partial
10/25	31	566 / 350 / 856	22,141 / 201,206	300 / Partial

Table 4: Model element and state-space reduction rates

Configuration	Model	Model Elements Reduction (%)			State Space Reduction (%)	
Scale	Tokens	Places	Transitions	Arcs	Nodes	Arcs
2/4	7	76.15	79.69	75.46	76.39	76.09
2/4	12	76.15	79.69	75.46	/	/
2/4	17	76.15	79.69	75.46	/	/
4/10	8	82.13	81.82	77.59	/	/
4/10	13	82.13	81.82	77.59	/	/
4/10	18	82.13	81.82	77.59	/	/
6/12	8	78.82	81.60	74.22	/	/
6/12	16	78.82	81.60	74.22	/	/
6/12	24	78.82	81.60	74.22	/	/
10/25	31	77.92	82.57	74.18	/	/

* Data marked with '/' indicates that the exact reduction rate is unavailable because the state-space computation timed out.

5 Conclusion

This paper proposes an automated HCPN modeling method for microservice orchestration workflows by formalizing Netflix Conductor specifications. Experimental results demonstrate that, under identical experimental conditions, this hierarchical strategy effectively mitigates the state-space explosion problem, achieving a 74%–83% reduction in model elements and an approximate 76% reduction in state-space size compared to traditional source-code-based models. Furthermore, validations using ASK-CTL formulas in CPN Tools indicate that the generated models can correctly reflect the workflow logic and automatically detect critical defects, such as data inconsistencies and business-data state discrepancies. Ultimately, this approach provides a reliable, low-cost formal verification framework for networked service systems. In future work, we plan to expand the current empirical evaluation to more comprehensive application scenarios, in order to more systematically assess the practical applicability and overall performance of the proposed framework

Acknowledgments

This work was supported in part by the Natural Science Foundation of Inner Mongolia under Grant 2026QC0213 and 2024QN06002; in part by Key Project of the Natural Science Foundation of Inner Mongolia Autonomous Region under Grant 2026ZD014; in part by the Scientific Research Start-up Fund for High-level Talents of Inner Mongolia University under Grant 10000-A260015/223; in part by the Inner Mongolia Science and Technology Program under Grant 2025YFSH0078; and in part by the Key Research & Development and Achievement Transformation Program Project of Inner Mongolia under Grant 2026YFSH0175.

References

[1] 2019. *Netflix's Conductor: A Microservices Orchestrator.* https://netflixtechblog.com/netflix-conductor-a-microservices-orchestrator-2e8d4771bf40 [Online].
[2] Matteo Camilli. 2020. Continuous formal verification of microservice-based process flows. In *European Conference on Software Architecture*. Springer International Publishing, Cham, 420–435.
[3] Luca Giamattei, Alessandro Guerriero, Rocco Pietrantuono, and Salvatore Russo. 2024. Automated Functional and Robustness Testing of Microservice Architectures. *Journal of Systems and Software* 207 (2024), 111857.
[4] Matthias Hilbrich and Frank Lehmann. 2022. Discussing Microservices: Definitions, Pitfalls, and their Relations. In *2022 IEEE International Conference on Services Computing (SCC)*. IEEE, Barcelona, Spain, 39–44. doi:10.1109/SCC55605.2022.00012
[5] Klaus Jensen, Lars M. Kristensen, Kurt Jensen, and Lars M. Kristensen. 2009. Formal Definition of Hierarchical Coloured Petri Nets. In *Coloured Petri Nets: Modelling and Validation of Concurrent Systems*. 127–149.
[6] Klaus Jensen, Lars M. Kristensen, and Lars Wells. 2007. Coloured Petri Nets and CPN Tools for Modelling and Validation of Concurrent Systems. *Int. J. on Software Tools for Technology Transfer* 9, 3 (2007), 213–254.
[7] Srinivas Kesarpu. 2025. Contract Testing with PACT: Ensuring Reliable API Interactions in Distributed Systems. *The American Journal of Engineering and Technology* 7, 6 (2025), 14–23.
[8] S. Naik and M. D'Souza. 2023. Detection of faults in microservices using petri nets. In *Proceedings of the 16th Innovations in Software Engineering Conference*. 1–5.
[9] Pawoot Rattanukul. 2023. Microusity: A Testing Tool for Backends for Frontends (BFF) Microservice Systems. In *2023 IEEE/ACM 31st International Conference on Program Comprehension (ICPC)*. IEEE, Melbourne, Australia, 74–78.
[10] A. M. Saucedo, G. Rodriguez, F. G. Rocha, et al. 2025. Migration of Monolithic Systems to Microservices: A Systematic Mapping Study. *Information and Software Technology* 177 (2025), 107590.
[11] Gokhan K. Soylu and Ozan Demirörs. 2023. An Exploratory Case Study: Using Petri Nets for Modelling Microservice-Based Systems. In *Proc. 49th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. 254–261.
[12] C. A. Sun, Y. Gong, M. Li, et al. 2024. Detecting inconsistencies in microservice-based systems: An annotation-assisted scenario-oriented approach. *IEEE Transactions on Services Computing* 17, 5 (2024), 2194–2209.